

MÓDULO 8: SEGURIDAD

INJECTION

OWASP TOP 10

BIG school

BIGSEO

GENERAMOS
NEGOCIO

Injection

Índice

- ¿Qué vamos a ver?
- ¿Qué es Injection?
- ¿En qué consiste?
- Ejemplos típicos
- Tipos de inyección (SQLi y XSS)
- ¿Qué puede hacer alguien que explote Injection?
- ¿Cómo detectarlo?
- ¿Cómo prevenirlo?
- ¿Cómo mitigarlo cuando ya está en producción?
- Checklist rápido para desarrolladores
- Conclusiones



Injection

BIG school

¿Qué vamos a ver?

- Una de las vulnerabilidades más clásicas y frecuentes.
- Permite ejecutar código o comandos no previstos al procesar entrada de usuario.
- Veremos qué es, ejemplos, tipos comunes (SQLi y XSS), impacto y prevención.

¿Qué es Injection?

- Sucede cuando datos externos son interpretados como instrucciones.
- El sistema ejecuta esos datos en lugar de tratarlos como contenido.

Regla esencial: no confiar en la entrada del usuario ni permitir que se interprete como código.

¿En qué consiste?

- No separar correctamente datos y comandos.
- Falta de validación o saneamiento del input.
- Uso de concatenaciones en consultas o instrucciones.
- Confianza en que el cliente limpia los datos.

Solución conceptual: tratar siempre los datos del usuario solo como datos.

Ejemplos típicos

- Formularios sin validación que pasan datos a componentes interpretados.
- Endpoints que usan datos de usuario dentro de consultas o comandos.
- Interfaces que reflejan datos sin escapado (XSS).
- Entradas enviadas a motores de plantillas o shells sin controles.

Estos patrones exigen aplicar validación, parametrización y escapado adecuados.

Tipos de inyección (SQLi y XSS)

SQL Injection: la entrada del usuario altera una consulta SQL.

- Riesgo: acceso, modificación o borrado de datos críticos.
- Prevención: parametrizar consultas, validar tipos y limitar privilegios en DB.

Cross-Site Scripting (XSS): contenido controlado por el usuario se ejecuta en el navegador de otros.

- Riesgo: robo de cookies, tokens, manipulación de la interfaz.
- Tipos: reflejado, persistente, y DOM-based.
- Prevención: escapado contextual, validación en servidor, uso de CSP.

¿Qué puede hacer alguien que explote Injection?

- Acceso o alteración de datos sensibles.
- Secuestro de sesiones o robo de identidad.
- Ejecución de comandos del sistema.
- Inyección persistente de código malicioso.
- Uso de la aplicación comprometida para campañas o fraudes.

Impacto alto: mínimo esfuerzo, consecuencias graves.

¿Cómo detectarlo?

Durante desarrollo:

- Revisar interpolaciones de datos en consultas o plantillas.
- Confirmar uso de APIs con parámetros seguros.

En pruebas/pentesting:

- Enviar entradas inesperadas y observar comportamiento.
- Análisis estático o dinámico para detectar patrones de riesgo.

En producción:

- Analizar logs con entradas anómalas.
- Detectar errores de parsing o contenido extraño en UI.
- Observar consultas o respuestas inesperadas.

¿Cómo prevenirlo?

Principios:

- Valida y normaliza datos en servidor.
- Separa datos de instrucciones.
- Escapa contenido según el contexto (HTML, JS, CSS, URL).

Buenas prácticas:

- Parametrizar consultas a la base de datos.
- Usar ORMs o query builders con manejo seguro de parámetros.
- Validación estricta (listas blancas). Escapar al renderizar y aplicar CSP.
- Evitar eval o ejecución dinámica de entrada del usuario.
- Cuentas de DB con mínimo privilegio. Separar endpoints y evitar almacenamiento de HTML sin sanitizar.

¿Cómo mitigarlo cuando ya está en producción?

- Restringir acceso al endpoint vulnerable.
- Revisar logs y determinar alcance.
- Aplicar validación o parametrización en código.
- Limpiar contenido malicioso almacenado.
- Rotar tokens y credenciales si hay compromiso.
- Aplicar reglas WAF mientras se parchea.
- Realizar auditoría y pruebas regresivas tras la corrección.

Checklist rápido para desarrolladores

1. ¿Validas y normalizas todas las entradas en servidor?
2. ¿Usas parametrización para consultas a la base de datos?
3. ¿Aplicas escapado contextual al renderizar?
4. ¿Evitaste eval o ejecución dinámica?
5. ¿Las cuentas de DB tienen privilegios mínimos?
6. ¿Hay controles de autorización y CSP activos?
7. ¿Existen pruebas que validen respuestas seguras ante entradas no esperadas?

Conclusiones

1. **Injection persiste, pero su prevención es simple: tratar datos como datos, nunca como instrucciones.**
2. **Buenos hábitos de validación, parametrización y escapado previenen errores graves.**
3. **Propón revisar un endpoint crítico y probar entradas inesperadas: es un paso práctico para mejorar la seguridad real del proyecto.**